智能运维平台

智能运维平台技术方案

技术架构 开源组件与工程实现

文档版本 V8.0 正式版

编制日期 2026 年 11 月 1 日

适用范围 非虚拟化智能运维

使用对象 技术管理 架构设计与研发评审

本方案说明平台如何将开源组件、业务规则与大模型接入实际运维流程，并通过资源身份、工具调用、证据核验与恢复检查形成完整的工程实现。

技术主线

以统一资源身份和来源记录建立数据基础

以固定工具和查询编译落实自然语言请求

以模型解释和诊断配方分别承担分析与正式判断

以人工处置和新事实读取完成恢复验证

目录

第 1 至 8 章介绍架构、开源组件与数据基础；第 9 至 15 章展开自然语言调查与诊断；第 16 至 20 章说明运行控制、处置、案例与部署；第 21 章提供源码导航。

01 方案定位与技术结论.....3
02 总体架构与程序协作.....4
03 AI 与工具协议开源组件.....5
04 数据观测与检查开源组件.....6
05 硬件观测与 Kubernetes 关联.....7
06 持久化与身份授权开源组件.....8
07 统一资源身份与关系图实现.....9
08 从原始观测形成异常与证据.....10
09 自然语言请求与调查上下文绑定.....11
10 模型推理与请求内容.....12
11 工具调用如何转成平台函数执行.....13
12 Agent 循环与 MCP 协议实现.....14
13 自然语言问数与固定查询编译.....15
14 故障链取证与规则模型协同.....16
15 正式根因分析的判定机制.....17
16 任务持久化与运行控制.....18
17 人工处置与恢复验证.....19
18 Pod 重启调查完整实现示例.....20
19 PVC 与 CSI 跨资源诊断示例.....21
20 部署配置与能力生效机制.....22
21 技术实现依据与源码导航.....23

01 方案定位与技术结论

本方案说明智能运维平台如何利用开源组件、资源关系、诊断规则和大模型完成 Kubernetes 环境中的问数、排障与处置验证，面向技术管理、架构设计和研发评审。技术说明以项目当前源码、接口 Schema、依赖锁定文件和组件登记记录为依据，覆盖非虚拟化资源、观测数据与人工处置链路。

平台将自然语言转化为可执行的查询和调查步骤，再以真实读取结果形成分析。 大模型承担问题理解、工具选择与结果解释；Go 平台承担资源定位、当前授权、数据读取、统计计算、规则判断和结果提交；Python 调查程序组织模型与平台工具之间的调用。各环节通过任务与证据记录关联。

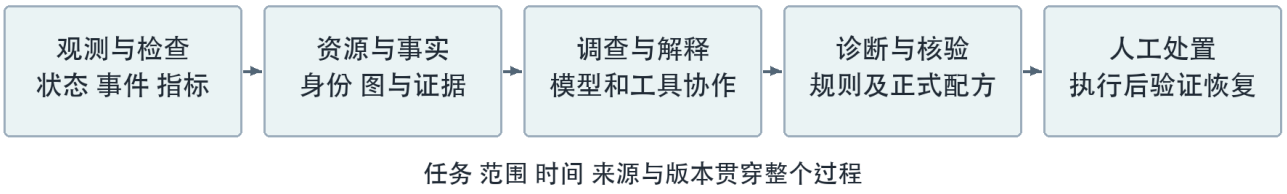


图 1 智能运维从观测到恢复验证的工作过程

智能运维的工程组成

平台首先采集 Kubernetes 状态、事件及已接入的指标、日志、网络和硬件信息，保留资源身份、来源版本与时间。资源图把 Pod、Node、PVC、StorageClass、CSIDriver 等对象连接起来，提供依赖查询与影响范围。固定检查程序将异常投影为标准 Finding，即带有资源、规则和发生记录的异常对象。

用户提出问题后，平台固定调查对象、授权范围与绝对时间窗。模型根据工具说明提出函数调用，平台执行查询并保存结果，调查程序再把结果交回模型。需要正式确认根因时，独立的诊断配方检查规定事实与证据条件；需要修改环境时，由具备权限的操作人员提交命令，并在执行后重新读取事实验证恢复。

方案形成的技术价值

自然语言入口复用已有运维数据和 API；工具目录将模型输出约束到明确业务操作；统计和规则由固定代码执行；每次读取、判断与处置保留可追溯记录。系统能力由代码、已发布配置和可用来源共同决定，本文不将组件存在或示例成立表述为生产环境验收完成。

02 总体架构与程序协作

系统由 Web 工作台、Go API 与 Worker、Python Investigator、模型服务和数据基础组件组成。API 接收业务请求并提供查询接口；Worker 推进采集、检查、调查与处置任务；Investigator 执行模型调查循环或固定取证。API 与 Worker 是平台的不同运行角色，业务模块在 Go 代码中组合。

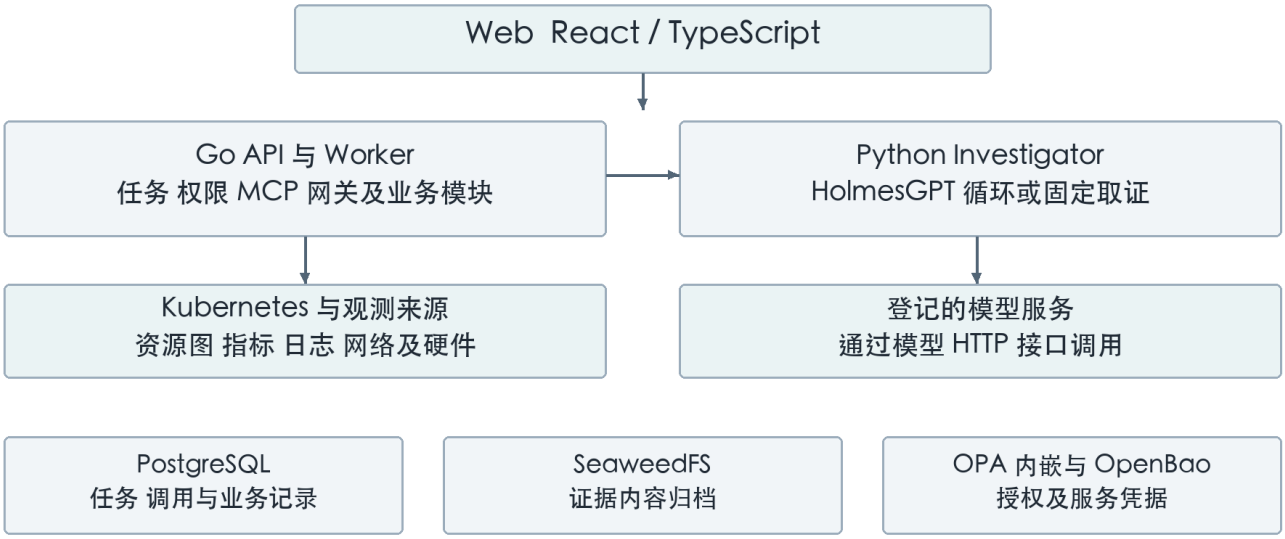


图 2 平台运行单元与主要数据组件

运行单元	接收的输入	处理与输出
Web 工作台	用户问题 资源选择 时间范围	展示任务进度 事实 关系与回答 提交追问和本人处置请求
Go API 与 Worker	身份 业务请求 已发布配置	生成任务 执行当前授权 管理 Step 与预算 调用来源并提交结果
Python Investigator	已绑定调查任务 模型配置引用	调用模型和 MCP 工具 返回结构化分析或固定取证结果
模型服务	messages 工具定义 输出限制	生成函数名与参数 或生成符合约定格式的分析

Python 通过模型 HTTP 接口调用推理服务，通过 MCP Streamable HTTP 调用 Go 工具网关，通过内部 Job API 管理 Step、续租与结果提交。它使用平台工具读取业务数据，数据库和 Kubernetes 客户端由平台侧业务模块管理。

PostgreSQL 保存控制记录和结构化业务数据，SeaweedFS 保存证据归档内容。Go 内嵌 OPA SDK 执行策略，Keycloak 提供 OIDC 身份，OpenBao 管理平台所需的密钥与工作负载凭据。模型端点与这些组件分别配置，模型服务的可用性和数据来源的可用性分别检查。

03 AI 与工具协议开源组件

AI 调查链使用 HolmesGPT 组织调用，使用 LiteLLM 适配模型接口，并通过官方 MCP SDK 连接平台工具。下表版本取自当前 Python 锁文件与 Go 模块文件；集成方式说明组件在本产品中实际承担的工作。

组件与锁定版本	接入方式	在产品中的具体使用
HolmesGPT 0.42.0	Python 库	构造 ToolCallingLLM 和 ToolExecutor 组织多轮模型与工具调用
LiteLLM 1.89.0	Holmes 依赖的模型调用层	依据模型 Profile 传递模型名 端点 凭据 输出限制与结构化输出参数
MCP Python SDK 1.28.1	Investigator 客户端库	初始化协议会话 读取工具目录 调用工具并接收 structuredContent
MCP Go SDK 1.1.0	Go 工具服务端库	注册业务工具的输入输出 Schema 处理标准 MCP 请求
Pydantic 2.13.5	Python 数据校验库	校验运行配置和模型 Profile 拒绝未知字段及不符合约定的配置
jsonschema 4.26.0	Python 输出校验库	检查模型最终 JSON 和引用字段是否满足输出合同

平台怎样使用 HolmesGPT

平台通过 PlatformTool 将 Go 返回的 MCP 工具定义转换为模型可读取的 function 定义。工具被模型选中后，PlatformTool 先向 Job API 分配调用 Step，再通过 MCPBridge 执行；已提交的相同调用可以读取既有结果。Holmes 收到工具结果后继续请求模型，直到形成回答或触及执行限制。

ToolCallingLLM 的构造参数设置 max_steps 为 8。平台预算仍独立控制实际模型调用次数、输入输出 Token、工具调用和返回数据量；最终输出修复也受同一预算管理，不能把 8 解释为一次任务绝不超过 8 次模型请求的独立承诺。

模型与推理服务的配置

模型是已经训练好的参数文件及其推理能力，模型服务对外提供 HTTP 接口。项目的自托管 Profile 记录模型标识、权重摘要、许可证标识、私网端点、上下文窗口、输出上限和准入证据。部署模板登记 Ollama 开发提供方和 llama3.1:8b-16k 开发模型，这些模板值不代表当前环境已启用或已满足回答质量要求。

04 数据观测与检查开源组件

数据组件为调查提供事实，检查组件将特定输入转换为标准异常。独立服务、命令行程序和内嵌源码分别集成，不能据一个开源项目名称推定平台运行了该项目的全部功能。

组件与登记版本	接入方式	在产品中的具体使用
VictoriaMetrics v1.116.0	指标服务与 HTTP 适配器	平台按固定模板读取 Pod CPU 内存 重启和 Node Ready 等已登记指标
VictoriaLogs v1.52.0	日志服务与 HTTP 适配器	按资源身份和时间窗读取日志样本 统计结果及内核日志记录
DeepFlow 7.2.0	可选网络来源与平台适配器	查询已启用的 L4 与 L7 数据 将后端端点映射为平台资源身份
K8sGPT v0.3.41	锁定命令行程序	固定检查 Pod Node PVC 输出 JSON 由平台接收为 FindingCandidate
Ariadne fa2232ce45c4	Go 库	基于 Kubernetes 对象引用 选择器和事件引用解析资源关系
kubernetes ontology 346236312685	选定源码与平台桥接	复用节点 边 诊断子图 证据排序与冲突表达 对接 Ariadne 图读取
Coroot v1.26.8	选定 Check 逻辑内嵌	对已取得的数值执行阈值判断 转换为统一检查结果
Node Problem Detector v1.35.3	kernel monitor 规则复用	对已取得的内核日志按规则匹配 生成节点相关检查结果

按平台边界使用检查组件

K8sGPT 的固定参数包含 analyze、Pod/Node/PersistentVolumeClaim 过滤、JSON 输出、关闭缓存和单并发，不开启 explain。这里使用它的分析器产生检查事实；自然语言解释由独立 Holmes 调查路径完成。

Coroot 在平台中复用 Check 值判断与相关数据结构，Node Problem Detector 复用内核日志规则。对应实现不等同于部署完整 Coroot 系统或 NPD DaemonSet。Ariadne 与 ontology 通过 Go 适配层协作，资源关系和诊断子图不依赖模型临时生成。

观测查询以来源注册、版本、覆盖范围和能力检查为条件。DeepFlow 为可选来源，L7 需要对应能力有效；本方案的网络分析范围为 L4/L7，不包含完整 Trace 调查链。

05 硬件观测与 Kubernetes 关联

硬件调查以已注册 Redfish 来源为当前主要读取路径。平台通过 BMC 取得服务器及部件状态，将其转为与 Kubernetes 资源可关联的硬件实体和事实，使节点侧问题可以沿真实清单关系追踪到物理服务器或部件。

组件与登记版本	接入方式	在产品中的具体使用
Gofish v0.26.0	Go Redfish 客户端库	读取 ComputerSystem 部件和日志等已限定对象保留原生 URI 与来源身份
Metal3 BMO v0.14.1	HardwareDetails 模型源码复用	统一厂商 CPU 内存 存储与网卡清单 接收 Redfish 投影后的硬件信息



图 3 硬件部件到 Kubernetes 资源的关系

Redfish 数据怎样成为诊断依据

Redfish 适配器对端点、HTTPS 凭据、请求数量、返回大小和对象范围执行限制。采集结果包含硬件实体、健康状态、诊断事实及降级信息。平台以规范化 UUID、序列号和来源清单解析身份，重复标识和无法确认的身份保留缺口；不能只凭显示名称关联服务器。

硬件事实经 Coroot 值检查或原生状态检查形成标准结果；节点内核日志经 NPD 规则匹配形成另一类事实。Metal3 在这里提供硬件数据模型，不代表平台通过 BMO 接管物理机置备生命周期。

从硬件关系进入 Kubernetes 调查

经过身份与清单绑定，平台建立部件属于服务器、服务器承载 Node、Pod 调度到 Node 的关系。依赖和影响查询使用这些关系定位相关资源，正式 DIMM 配方再检查部件健康、不可纠正 ECC 等规定事实与完整路径。

BMC 告警、节点日志与应用异常分别保留来源和时间。相关现象只有满足对应配方的因果条件、证据独立性与完整性要求后，才能形成正式根因；某个部件状态异常本身不等于该节点所有 Pod 均受影响。

06 持久化与身份授权开源组件

组件与登记版本	接入方式	在产品中的具体使用
PostgreSQL 17.11	独立数据库 pgx 访问	保存任务 Step 预算 会话 调查绑定 资源历史 Finding 结果索引与审计记录
SeaweedFS 4.47	S3 兼容归档服务	保存证据及执行输出的归档内容 由数据库保留摘要 来源与对象引用
Keycloak 26.7.4	OIDC 身份服务	提供用户身份和登录声明 平台结合当前租户角色与资源范围授权
OPA Go SDK 1.21.0	Go 进程内嵌 Rego 求值	将当前主体 角色 资源与操作交给策略判断 用于业务 API 与工具调用授权
OpenBao 2.7.0	密钥与身份服务	通过平台适配器管理 Transit 签名 工作负载 PKI 与处置所需 SSH 凭据
Ansible Runner 2.4.3	隔离的 Command Runner	在处置路径执行已绑定命令 支持锁定的 Kubernetes Bash 与 Ansible 传输

数据库与归档服务的分工

一次工具读取对应一个 Step，数据库保存其输入摘要、状态、业务结果与引用。需要归档的证据内容写入 SeaweedFS，平台保存内容摘要及归档状态。重新读取时，平台检查当前权限、来源版本和内容完整性，不能仅凭一个已有记录 ID 将其作为有效诊断依据。

用户身份与程序身份的分工

用户身份决定其能查询和处置哪些资源；程序身份决定 Worker、Investigator 与内部网关能否互相调用。内部调用使用 mTLS，并绑定签名任务上下文、租约和调用凭据。Go 网关在每次工具执行时检查当前权限，任务创建时取得权限不代表后续调用一直获准。

OPA 在这里以内嵌 SDK 执行策略。OpenBao 提供密码材料与身份能力，具体业务范围仍由平台授权逻辑决定。命令执行使用单独的处置流程及 Runner，调查工具目录保持只读，模型生成的分析不会直接成为修改请求。

开源复用与平台实现的关系

平台自行实现来源绑定、统一资源身份、语义工具、调查任务、预算账本、证据引用检查、诊断配方提交与恢复判定。开源库提供协议、图解析、检查或基础存储能力，产品通过这些业务约束把组件连接为一致的运维流程。

07 统一资源身份与关系图实现

资源定位从租户、集群、资源类型和不可变 UID 出发。平台用 Canonical ID 标识一个具体资源实例，并保存 namespace、名称、resourceVersion、观测时间及来源。Pod 删除后同名重建会获得新的 UID，历史事件和调查结果必须继续指向原实例。

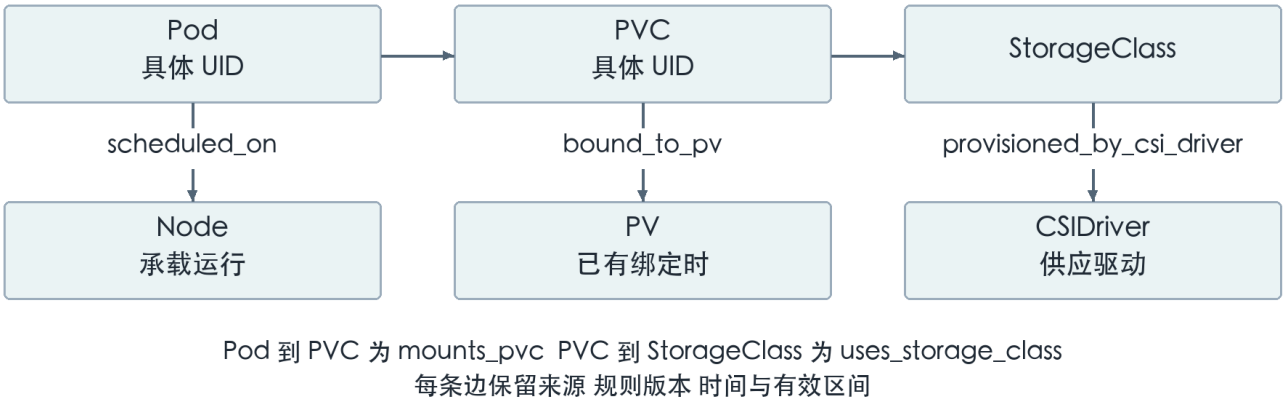


图 4 资源实例与诊断关系示意

Ariadne 如何解析资源关系

Go 图适配器把 Kubernetes 对象交给 Ariadne 的 StructuralResolver、SelectorResolver 和 EventResolver，解析结构引用、标签选择器和事件引用。平台再将解析结果投影为 managed_by、scheduled_on、mounts_pvc、bound_to_pv、uses_storage_class 等业务关系。

平台对涉及 UID 的引用再次检查。例如 Event 的 involvedObject UID 必须与目标 UID 一致，PV 的 claimRef UID 不能连接到同名替代 PVC。每条关系保留来源注册、解析规则版本、观测时间和有效区间，查询同时限定租户、资源范围、深度和返回数量。

ontology 如何参与诊断查询

ontologyReader 在 Ariadne 快照上实现 GetNode、Neighbors 等读取接口，把平台资源映射为诊断节点和边。依赖查询沿依赖方向展开，影响范围沿被依赖方向展开；事件可作为观测依据，但事件引用不直接作为故障传播边扩展影响范围。

图回答资源如何关联，规则和来源数据回答这些资源当前发生了什么。模型读取平台返回的关系并组织解释。关系存在本身不证明因果，例如 Pod 挂载 PVC 只能建立排查路径，确认 CSI 供应故障还需要规定的状态、事件与来源身份。

实现入口为 internal/graph/ariadne_adapter.go、ontology_bridge.go 与 internal/upstream/ontology。图快照的新鲜度、完整性和来源信息会进入后续统计与诊断判断。

08 从原始观测形成异常与证据

Go 采集器通过 Kubernetes List 与 Watch 获取资源及事件，记录 resourceVersion、List 完成时间、Watch 连续性和来源状态，更新当前资源图并保存历史投影。指标、日志和网络通过注册适配器读取；原生 PodMetrics 路径读取 metrics.k8s.io，并核对 Pod、Node 与容器身份。采集时间、业务时间和数据覆盖分别保留，当前状态与故障窗内的历史观测分别处理。

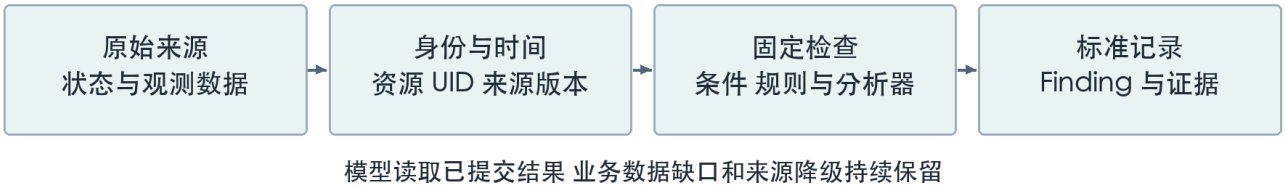


图 5 原始观测形成标准异常与证据

异常检查怎样执行

Kubernetes 原生检查程序读取 status、Condition 和 Event。例如 Node Ready=False 形成 NotReady 信号，Ready=Unknown 单独保留未知状态，PVC Pending 形成未绑定信号。K8sGPT 输出经锁定格式解析后产生同一平台合同下的候选异常；Coroot 值检查和 NPD 日志规则通过各自适配器输出检查结果。

Worker 绑定实际租户、来源、资源 UID、规则版本与时间，将 FindingCandidate 转为标准 Finding 及 occurrence，即该异常的一次发生与状态变更记录。重复异常、恢复与同名重建资源通过稳定身份区分，模型生成的一句话不进入这一采集检查流程。

三类依据分别保存

记录类型	保存内容	后续如何使用
资源观测与 Step	状态投影 Events 工具输入输出 时间与来源	支持历史对比 资源检查和助手引用
QueryResult 与 Fact	实际统计表 数值 单位 覆盖范围和结果摘要	支持问数回答及故障窗内的数据判断
Evidence	已绑定来源的证据及归档摘要	满足相应配方和完整性条件后参与正式诊断

平台保留缺失、截断、来源降级和覆盖不完整信息。未查到日志、未出现新异常或没有观测记录，都不能自动转换为资源健康。不同工具中的同一底层事实也不能重复计算为独立证据。

09 自然语言请求与调查上下文绑定

自然语言输入进入平台后，先成为有范围的业务任务。平台区分 data_query 问数、resource_troubleshooting 资源排障和 incident_followup 事件追问。资源选择、时间、授权和模型配置先在业务接口中绑定，再交给调查程序使用。

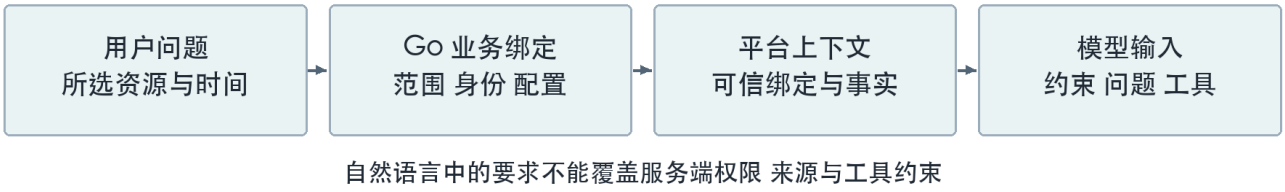


图 6 自然语言请求进入已绑定的调查任务

问题如何对应具体资源与时间

“这个 Pod 为什么一直重启”需要一个确定的 Pod 实例。页面选择或用户确认得到的目标 ID、当前集群和 namespace 进入任务；“过去一小时”在提交时解析为绝对起止时间。故障窗、基线窗和恢复窗分别绑定，工具接收 windowRole 后由服务端解析具体时间。

平台保存可信绑定字段和用户原文。可信字段来自身份、资源选择和已发布配置；用户问题、日志、事件正文和历史回答作为数据提供给模型。它们不能覆盖工具可见范围、来源凭据、时间约束或策略。平台在必要上下文不完整时要求补全或返回明确缺口。

上下文怎样进入模型

上下文是本次模型请求中附带的内容，包含平台约束、已绑定对象、当前问题、已提交工具结果和允许引用的记录。调查程序先通过 get_assistant_context 读取平台上下文；排障和事件追问还会按任务类型预读必要事实，再进入模型分析。

模型在请求之外不会自动知道集群状态。每轮所需事实由程序读取并放入 messages；后续追问形成新的调查轮次和任务记录，历史内容通过平台校验后再作为背景。这个过程不更新模型权重，也不依赖模型服务自行保存运维会话。

助手输出、平台附加的来源 Step 和正式 RCA 分别保存。模型可以解释已有正式结论，但不能自行生成已执行检查、证据 ID 或平台拥有的引用字段。

10 模型推理与请求内容

大模型在本产品中表现为一个推理 HTTP 服务：程序发送输入，服务利用已训练的权重生成文本或结构化函数请求。模型权重保存训练得到的参数；推理是在固定参数下处理一次输入。当前调查链通过提供业务上下文使用模型，没有在调查过程中训练或微调模型。

一次模型请求包含什么

下面用 Python 数据结构展示模型请求的主要字段。变量由程序填充，messages 和 tools 随任务变化，模型标识与输出上限来自已准入的 Profile。实际 HTTP 适配由 Holmes 与 LiteLLM 完成。

```
request = {
    "model": profile.modelID,
    "messages": [
        {"role": "system", "content": policy},
        {"role": "system", "content": trusted_context_json},
        {"role": "user", "content": committed_results_json},
        {"role": "user", "content": question},
    ],
    "tools": allowed_function_definitions,
    "max_tokens": profile.outputTokenLimit,
}
```

policy 即提示词中由平台编写的行为要求，说明只读工具、数据可信性和回答格式。trusted_context_json 描述已绑定目标与范围。committed_results_json 是平台已保存结果的模型可见投影。tools 描述可调用函数的名称、用途和参数 Schema，而不是函数源码或访问凭据。

Token 与结构化输出如何控制

模型把输入与输出拆为 Token 处理和计量，Token 不能直接等同于汉字数。上下文窗口限制一次请求可容纳的总内容，输出上限限制生成长度。BudgetedLLM 在请求前预留预算，在请求后结算实际用量；结果和调用状态交给平台保存。

最终回答需要满足指定 JSON 输出 Schema。平台使用提供方支持的 json_schema，或采用 json_object 并在提示词内提供 Schema；返回后仍执行本地结构校验和引用校验。格式合格只表示结构可处理，回答中的事实仍必须对应成功提交的工具结果。

11 工具调用如何转成平台函数执行

工具调用分为模型返回请求和平台执行函数两个阶段。模型看到的工具定义由 MCP 目录的名称、描述与输入 Schema 转换，PlatformTool.get_openai_format 负责生成 function 结构。

提供给模型的函数定义

下面以 resource_history 为例，保留实际参数名与时间角色，省略 Canonical ID 的外部 Schema 引用，用 string 展示其类型。完整合法性检查仍使用平台登记的 Schema。

```
{
  "type": "function",
  "function": {
    "name": "resource_history",
    "description": "读取绑定资源在指定时间角色下的历史观测",
    "parameters": {
      "type": "object",
      "properties": {
        "canonicalId": {"type": "string"},
        "windowRole": {"enum": ["fault", "baseline", "recovery"]}
      },
      "required": ["canonicalId", "windowRole"],
      "additionalProperties": false
    }
  }
}
```

收到函数请求后怎样执行

模型返回的 tool_calls 中包含函数名与 JSON 字符串参数。下列代码展示解析参数、分配 Step、复用既有结果或发起 MCP 调用的衔接，为流程示意。

```
args = json.loads(tool_call.function.arguments)
allocation = job_api.allocate(tool_call.function.name, args)
if allocation.get("committed"):
    result = read_committed_result(allocation)
else:
    result = bridge.call(tool_call.function.name, args, allocation)
```

Go 工具网关依次检查程序身份、签名上下文、租约、工具目录、当前权限、输入 Schema、资源与时间范围及预算，随后调用业务 API。结果脱敏并校验输出合同后提交 Step，提交成功才返回调查程序。resource_history 读取历史观测与归属事件，不会执行模型提供的命令。

12 Agent 循环与 MCP 协议实现

Agent 是驱动模型与工具反复协作的调查程序：调用模型，执行函数请求，再把结果放回消息。HolmesGPT 的 ToolCallingLLM 提供循环，平台适配层处理任务、授权和提交。

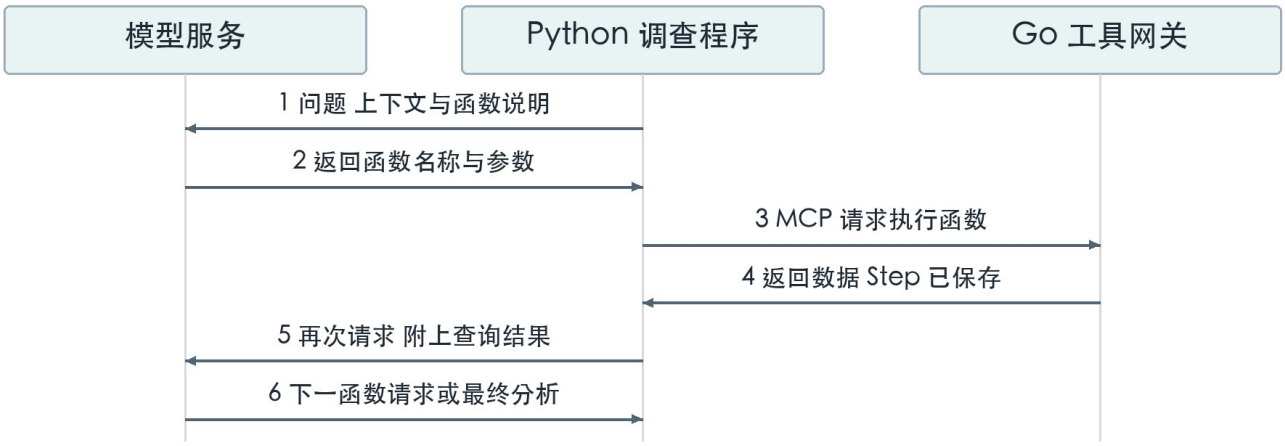


图 7 模型 工具和调查程序的调用顺序

循环的程序逻辑

下面为便于阅读的流程伪代码。模型返回 tool_calls 时执行读取并继续下一轮；返回最终内容时校验输出并提交。实际实现还包括任务恢复、预算结算、错误处理及最多一次受预算约束的输出修复。

```
messages = build_messages(job, committed_steps)
for turn in admitted_rounds:
    reply = call_model_with_budget(messages, tools)
    if reply.tool_calls:
        results = execute_platform_tools(reply.tool_calls)
        messages = append_tool_results(messages, reply, results)
        continue
    answer = validate_output_and_references(reply.content)
    return complete_result(job, answer)
finish_with_execution_state(job)
```

MCP 在其中传递什么

MCP 连接 Python 调查程序和 Go 工具服务，模型接口连接调查程序和模型服务。Python 客户端通过 Streamable HTTP 初始化 MCP 会话，校验协议版本 2025-06-18，通过 list_tools 获取目录，通过 call_tool 传递函数名与参数，接收 structuredContent。

MCP 元数据中的 ops/stepId 与 ops/jti 绑定调用 Step 和凭据。官方 SDK 处理协议，平台代码执行授权与业务函数，适配层衔接模型返回的函数请求和实际 MCP 调用。

13 自然语言问数与固定查询编译

问数使用模型表达查询意图，Go 服务执行数据筛选与计算。例如“所选范围内，各健康状态的 Pod 有多少个”，模型构造 QueryIntent，指定数据集、指标、维度和过滤条件；服务端从当前身份与页面选择取得执行范围。

```
{
  "schemaVersion": "query-intent/v1",
  "datasetId": "resources.current",
  "operation": "aggregate",
  "metrics": ["resource_count"],
  "dimensions": ["health"],
  "filters": [{"field": "kind", "operator": "eq", "value": "Pod"}],
  "sort": [],
  "limit": 10
}
```

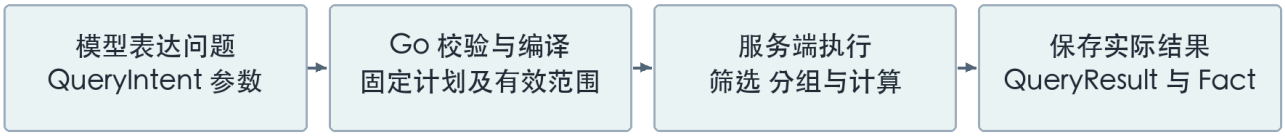


图 8 自然语言意图与服务端查询计算

查询编译如何落实

DecodeIntent 拒绝未知字段及非法结构，Compile 按固定目录检查数据集、operation、指标、维度和过滤字段。resources.current 路径要求显式非虚拟化 kind，构造图快照的确定查询计划，再按实际资源分组和计数。结果只有在快照新鲜、来源与完整性满足要求时才能支持完整统计，缺失来源不按零处理。

指标与日志经 Victoria 适配器处理，服务端依据登记的数据集和固定模板编译查询，检查来源资格、窗口、资源 UID 和覆盖范围。DeepFlow 经独立适配器使用固定 L4/L7 模板和后端身份映射。模型不能在 QueryIntent 中指定任意 SQL、PromQL、后端端点或更大的授权范围。

结果怎样返回模型

平台保存 QueryResult 与 Fact，保留数据表、数值、单位、时间、来源和摘要。模型收到已提交结果后生成自然语言说明，并引用真实结果。统计值由程序计算，模型负责组织表达；未执行查询的问数结果保留 query_not_executed 等状态，不能用模型自行推算的数字替代。

14 故障链取证与规则模型协同

资源排障包含模型组织调查和固定故障链取证两种执行方式。前者允许模型在给定工具范围内决定后续读取；后者由 Python 按任务要求执行预定义步骤。两种方式共用 MCP、Step、当前授权和预算，并使用 Go 的固定评估逻辑整理已读取的事实。

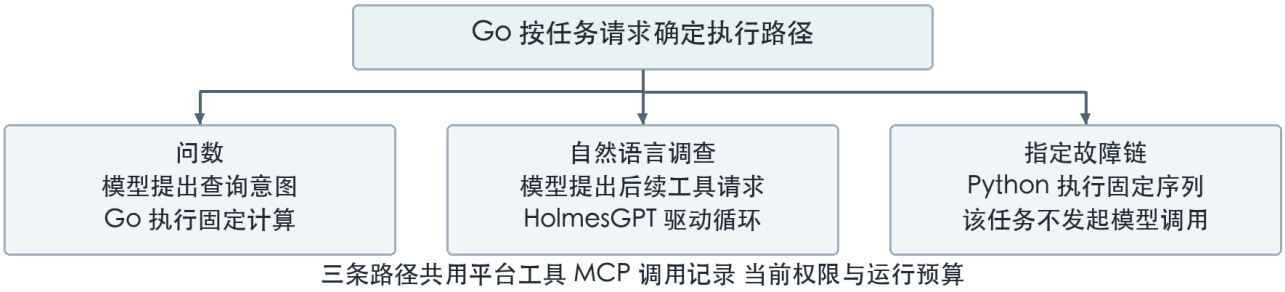


图 9 问数 模型调查与固定取证分流

固定取证怎样实现

executionMode 为 evidence_only 时，Worker 调用 fault_chain.collect。程序读取 get_assistant_context 中的 faultChainRequest，按目标类型读取历史、Finding 和变更，再根据已启用能力读取适用指标、日志、关系和关联资源。Pod、Node 和 PVC 的取证集合不同，PVC 不套用 Pod CPU 或内存查询。

程序预留评估和反证读取预算，限制关联扩展与结果规模，汇总成功 Step ID，调用 evaluate_fault_chain。Go 从同一任务中加载已提交 Step，重新检查权限、来源和结果引用，再生成 ResourceAssessment、关系及缺口。该分支不发送模型请求，但仍经过当前 Worker 的模型 Profile 和任务准入检查，不能表述为模型服务故障时自动切换的通用降级机制。

固定规则与模型各自判断什么

输入事实	平台规则形成的判断	模型在此基础上的工作
同一 Pod UID 两次观测的 restartCount 增加	记录 Pod 重启增加 并引用对应 Step	结合退出原因 指标与日志说明后续检查方向
terminated 记录的 reason 为 OOMKilled 且 finishedAt 在故障窗内	记录容器 OOM 退出事实	结合内存与限制信息分析 不直接认定泄漏或 Node 内存耗尽
故障窗内存在 firing 的 Finding occurrence	记录已提交异常发生及原始来源	关联同窗状态与关系 区分现象 原因假设和证据缺口

模型能够解释和补充调查步骤，固定规则提供可复核的条件核验。助手故障链评估与正式 RCA 采用不同的提交合同，助手分析不会直接写成正式根因。

15 正式根因分析的判定机制

正式根因分析使用版本化 Recipe，即可执行诊断配方。配方定义入口资源、图查询计划、必需事实、排除条件、工具集合、预算和恢复检查。当前内置正式根因配方覆盖 DIMM 硬件故障、PVC 与 CSI 供应故障、Node 运行故障；Pod、Deployment 和 Service 相关配方还用于原生恢复检查，不据此宣称任意故障都能自动确认根因。

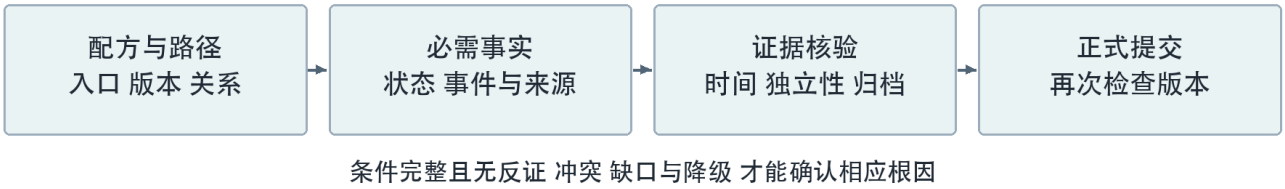


图 10 正式根因评估与提交条件

以 PVC 与 CSI 供应故障为例

平台检查 PVC Pending、匹配的 CSI 错误，以及 Pod 挂载 PVC、PVC 使用 StorageClass、StorageClass 对应 CSIDriver 的关系。CSI 错误的 reportingController 必须与关系图中的驱动身份匹配；普通 ProvisioningFailed 事件仅证明供应症状，不能单独证明目标驱动故障。

confirmed 的形成条件

rca.Evaluate 对照内置版本核验配方，检查证据时间可靠、与故障窗相交、未晚于评估时间且符合 300 秒新鲜度限制，要求归档可验证、内容摘要一致，并按独立性分组去重。图路径必须新鲜、完整，关系来源和有效区间满足要求。

全部必需事实存在、至少两个有效独立支持组且没有反证、缺口、冲突或降级时，评估结果才能成为 confirmed。条件不足时保留实际状态与缺口。正式提交时，Repository 再检查当前配方激活、图与来源版本、证据归档及 Finding 版本等，形成带版本依据的正式记录。

模型输出中的“可能原因”、文字语气或自报置信度不参与这些确认条件。证据排序分值服务于排序，不能解释为根因正确率。配方覆盖范围和数据条件决定正式诊断的适用边界。

16 任务持久化与运行控制

调查任务是 PostgreSQL 中的业务 Job，Step 是一次模型请求或工具调用记录；它们与命令执行使用的 Kubernetes Job 属于不同对象。平台把执行状态和预算放在数据库，Python 进程保存当前调用过程，进程内消息不是任务的唯一依据。

持久记录	保存内容	运行时用途
会话与调查轮次	用户问题 资源和时间绑定 历史引用	固定本轮上下文 追问形成新任务
调查 Job 与租约	任务身份 状态 配置引用 执行归属和有效期	Worker 推进任务 并限制过期执行方
Step 与参数摘要	函数名 输入摘要 状态 成功结果或失败原因	复用已提交读取 绑定引用 记录实际执行
预算账本	调用前预留 调用后消耗 剩余额度	约束模型次数 Token 工具 数据量与执行过程
回答与事实引用	通过校验的 JSON 来源 Step 查询结果和缺口	展示回答 支持复核和后续追问

调用与恢复怎样衔接

工具执行前，Job API 分配 Step 和调用凭据。已提交的等价调用返回已有 Step，Python 读取其结果；需要新执行时，MCP 网关以租约和凭据校验后开始调用，成功结果提交数据库后再返回。接管过程可以基于已提交读取重建模型上下文，过期执行方受到租约检查限制。

模型预算与失败怎样处理

BudgetedLLM 在实际请求之前为模型 Step 预留输入输出预算，用 requestDigest 绑定请求。成功时按提供方返回用量结算；无法确认用量或调用失败时按约定保守计量。Profile 固定超时与零重试配置，输出修复最多一次且仍记入预算。

模型超时、鉴权失败、上下文超限、限流与不可用被转换为明确类别。来源查询失败、结果截断和权限撤销也保留各自状态。平台不会把这些执行失败改写为“未发现异常”，任务状态和业务判断分别展示。

17 人工处置与恢复验证

调查流程输出事实、原因分析、缺口与建议检查。修改环境进入独立的 Command Execution 流程，由操作人员输入并确认具体命令，平台绑定执行目标、UID、当前身份、风险及相关事件，检查所需身份验证与权限后调度隔离 Runner。

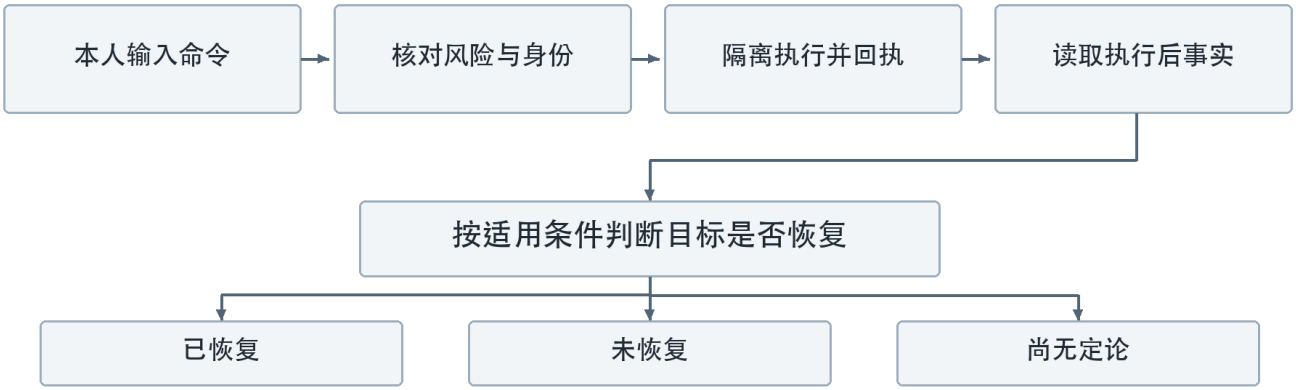


图 11 人工处置与执行后的恢复检查

Command Runner 怎样执行

平台使用一次性 Kubernetes Job 承载 Runner，运行锁定的 Bash、kubectl 或 Ansible Runner 路径。Runner 领取已绑定命令，使用规定的目标凭据，限制执行超时和输出大小，通过内部回调持续提交输出与结束状态。执行输出归档并保留摘要，模型调查进程不持有这条写入执行能力。

恢复判定怎样实现

命令退出码描述命令是否执行成功，恢复判断需要执行结束后的新事实。平台按目标类型选择已发布的 PostCheck 配方，读取目标状态、关联资源与近期 Warning，验证同一资源身份、授权、来源、完整性和采集时间，再形成 resolved、not_resolved 或 inconclusive。

目标支持声明覆盖 Node、PVC、Pod、Deployment、Service 与 EndpointSlice，实际执行还需要合格来源、配方激活和归档依据。Pod 检查涉及其状态和关联事实；PVC 检查结合供应路径及使用该存储的资源。来源不可用、身份变化或观测不足时，结果保留 inconclusive。

带有验证目标的处置还要满足对应目标；回滚任务同时检查目标规格摘要。新异常会影响恢复结论。处置回执、执行输出、恢复事实与诊断记录保持关联，便于核对“做了什么”和“问题是否消失”。

18 Pod 重启调查完整实现示例

本例说明实际程序协作方式，观测值为机制示例，不代表一次已完成的现场调查。用户在工作台选择一个 Pod，询问“这个 Pod 在故障窗内为什么反复重启”。平台将问题绑定到具体 UID、namespace 和绝对时间窗。

执行阶段	程序实际处理	形成的结果
任务建立	Go 检查权限 固定目标 窗口和模型 Profile	新的资源排障 Job 及可信绑定
初始取证	Investigator 经平台工具读取上下文和必需资源事实	成功 Step 包含历史 Finding 和变更
模型提出读取	Holmes 将问题 事实和函数定义发给模型	模型返回指标 日志或关系读取请求
平台执行查询	Go 网关校验调用 读取注册来源 保存结果	QueryResult Fact 资源关系及来源状态
规则与解释	固定规则核对重启和退出记录 模型解释已读事实	支持依据 可能原因 反证与下一步检查
提交与展示	平台校验回答结构 当前引用和范围	可打开依据的调查回答与执行记录

以 OOMKilled 记录说明判断深度

假设同一 UID 的容器 restartCount 从 3 增至 6，故障窗内存在 reason=OOMKilled 的终止记录。规则记录重启增加和 OOM 退出。模型在这些事实上提出内存趋势、资源限制、关联变更和应用日志等检查，并通过可用工具取得数据后继续解释。

若指标展示内存接近容器限制，可作为解释背景；具体泄漏需要更充分的应用证据。Node MemoryPressure、节点内核 OOM 和容器 OOM 退出是不同来源事实，不能由一个 OOMKilled 字段互相替代。日志覆盖缺失时，回答明确保留该缺口。

用户继续追问与采取措施

用户追问“最近变更是否相关”，平台创建新一轮任务，继承经校验的背景并重新绑定查询范围。若操作人员决定调整配置，进入本人处置流程；执行结束后，平台按适用配方读取新事实验证 Pod 和相关资源。原有累计 restartCount 不会因进程恢复清零，恢复必须结合新窗口与新观测判断。

19 PVC 与 CSI 跨资源诊断示例

用户询问“Pod 一直 Pending，是否与存储有关”。平台先读取调度状态和 Events，并通过资源图找到 Pod 使用的 PVC、StorageClass 与 CSIDriver。调查由具体资源与关系展开，不能仅根据问题中的“存储”一词确定故障原因。

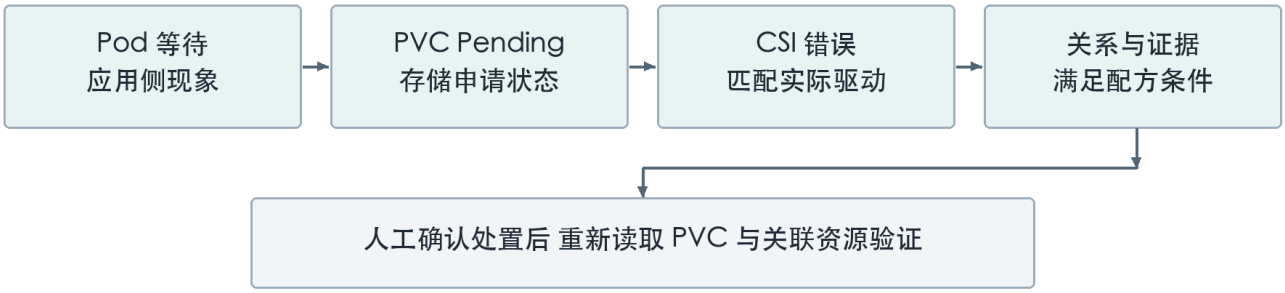


图 12 PVC 与 CSI 调查及恢复验证

从现象到因果条件

Pod 的等待或挂载失败是应用侧现象；PVC Pending 是存储申请状态；ProvisioningFailed 记录供应失败；驱动身份匹配和正确关系路径才把这些事实连接到指定 CSI 供应故障。必要时调查程序读取 PVC 历史、变更、原生只读检查和已授权的关联资源。

模型说明事件内容并选择后续读取，固定故障链评估汇总已提交 Step。进入正式 RCA 时，pvc-csi-failure 配方独立检查 Pending、匹配 CSI 错误和图路径，再检查证据独立性、时间、来源与归档。缺少任一必需条件时，正式结果保留缺口，不能将助手叙述直接写成 confirmed。

处置后验证关联恢复

操作人员确认具体处置后，Runner 执行并返回回执。平台重新读取 PVC、相关 PV、Pod、StorageClass、CSIDriver、VolumeAttachment 等适用对象和事件，按已启用配方核对供应、挂载和使用路径。仅有命令成功或 PVC Bound 不能替代全部已声明的验证条件。

本案例也说明智能运维的组合方式：图解析确定排查对象，状态与事件提供事实，工具程序执行读取，模型组织调查与解释，配方形成正式判断，处置与 PostCheck 完成结果验证。各阶段使用同一套资源身份和来源记录衔接。

20 部署配置与能力生效机制

部署按运行单元配置 Web、Go API、Go Worker、Python Investigator、数据库、归档和身份组件。模型服务单独提供端点，指标、日志、DeepFlow 与硬件来源按需要注册。源码中存在适配器、组件版本已登记和某个环境实际可查询，分别属于不同条件。

模型调查生效所需配置

Go 与 Python 必须使用匹配的模型 Profile 引用和摘要，模型支持工具调用与结构化输出，端点、凭据、上下文与输出限制满足合同。当前代码还定义单独的 DeepSeek 托管接口合同，固定校验端点、模型标识和计量方式；托管模型不归入开源组件。Investigator 的内部服务地址、MCP 地址、Job API 地址、工作负载证书与信任配置一致，调用才能进入任务执行。

数据查询生效所需配置

来源注册保存实际端点、类型、凭据版本及数据范围映射。平台检查来源资格、采集器与查询模板版本、有效期和窗口覆盖，再向任务暴露相应能力。指标、日志、网络按数据集和操作分别判断；能读取样本不等于任意聚合已生效，L4 可用也不等于 L7 自动可用。

新增一种调查能力的实现步骤

扩展从业务合同开始：定义数据集或工具输入输出 Schema，确定资源身份、时间与权限语义；实现服务端适配器和固定计算；接入来源资格、结果持久化及引用校验；补充工具目录与模型说明，最后进行对应来源和运行路径验证。新增正式根因类型还需实现并激活新的配方版本与证据谓词，不能只修改提示词。

本方案范围与验证口径

当前说明限定非虚拟化智能运维，KubeVirt 与 CDI 保持延期，完整 Trace 不在本方案实现范围。代码合同、来源能力验证、运行机制验证与模型回答质量分别评价。生产部署还以实际 Profile、组件准入、离线交付及环境验收记录为依据；本文不提供未经运行验证的准确率、性能或自动修复承诺。

21 技术实现依据与源码导航

本方案以 2026 年 11 月 1 日工作区代码为依据。以下路径相对于 platform 目录；组件登记版本和程序依赖锁定共同说明使用来源，具体环境的已发布配置决定运行集合。读取顺序可先看任务与工具，再看数据、诊断和处置。

技术主题	主要源码位置	可核对的实现
AI 调查与模型	services/investigator/src/investigator/ 下的 holmes_adapter.py assistant_binding.py model_profile.py budget.py	工具适配 提示词 可信绑定 Profile 结构化输出与预算结算
MCP 与固定取证	同目录 mcp_bridge.py fault_chain.py worker.py	协议连接 evidence_only 分流 固定步骤与成功 Step 汇总
Go 工具网关	internal/investigation/ 下的 mcp/server.go 与 tools/catalog.go	目录 Schema 当前授权 范围 预算 执行与提交
历史与故障链	internal/investigation/ 下的 resource_history.go resource_correlation.go fault_chain_committed.go	UID 历史读取 条件规则 已提交结果再校验
资源关系	internal/graph/ 下的 ariadne_adapter.go ontology_bridge.go	图快照 关系解析 身份约束 依赖与影响查询
问数与来源	internal/analytics/ 下的 compiler.go victoria.go 与 internal/integrations/deepflow	QueryIntent 编译 固定查询模板 来源与覆盖校验
原生检查与硬件	internal/inspection 及 internal/integrations/ 下的 k8sgpt 与 redfish	Kubernetes 与硬件状态 K8sGPT Coroot NPD 与 Redfish 读取
正式 RCA	internal/rca/ 下的 recipe.go engine.go repository.go	内置配方 必需事实 独立证据 正式评估与提交
处置与验证	internal/action/ 下的 runner.go postcheck_report.go postcheck_support.go	隔离命令执行 回调与归档 原生恢复判断
组件与部署合同	go.mod services/investigator/ 下的 pyproject.toml 与 uv.lock bundle/component-catalog.yaml deploy/profiles/kubernetes-containerd.yaml	精确依赖 组件来源 开发模板与基础组件配置

完整组件来源、提交摘要和归档材料以组件登记与依赖锁定为准。选定源码、规则复用与独立服务的集成边界见组件章节，登记记录不替代实际环境的能力检查。